Building an on-demand feature with a Python UDF

Let's also create a second feature table using on-demand feature engineering. Focus on the `CH5-04-Building Maximum Price Feature Table` notebook. We briefly introduced a scenario in an early section of this chapter requiring us to calculate a transaction's difference from the maximum price on the fly. This could be useful for modeling. To get the difference as an on-demand feature, we do the following:

1. Calculate the maximum price per product over a rolling window. We begin with creating `time_window`. We want the maximum price in the last three minutes:

```
fe = FeatureEngineeringClient()

time_window = F.window(
    F.col("TransactionTimestamp"),
    windowDuration="3 minutes",
    slideDuration="1 minute",
).alias("time_window")
```

Figure 5.29 – Creating a time window to calculate the rolling maximum prices by product

2. Now that we have a window, we can calculate and save that maximum price to a DataFrame. Usually, the value of a maximum price doesn't drastically change, so hourly is an appropriate timeframe. We add a new time column called `LookupTimestamp` for joining on.

```
max_price_df = (
  raw_transactions_df
    .groupBy(F.col("Product"),time_window)
    .agg(F.max(F.col("Amount")).cast("float").alias("MaxProductAmount"))
    .withColumn("LookupTimestamp",
                F.date_trunc('minute',
                             F.col("time_window.end") + F.expr('INTERVAL 1 MINUTE')))
    .drop("time_window")
)
```

Figure 5.30 – Creating a DataFrame feature table of product maximum prices